

Assignment 8

Author: Lan Stare

Date: 7. 11. 2025

Question 1

1. ADS Assignment 8

a) $T(n) = \begin{cases} 51 & ; n=1 \\ 5T(\frac{n}{5}) + 13 & ; n=5^k \text{ for } k > 0 \end{cases}$

$T(n) = 5T(\frac{n}{5}) + 13 =$ $T(\frac{n}{5}) = 5T(\frac{n}{5^2}) + 13$

$= 5(5 \cdot T(\frac{n}{5^2}) + 13) + 13 =$

$= \dots =$

$= 5^k \cdot T(\frac{n}{5^k}) + 13 + 5 \cdot 13 + 25 \cdot 13 + \dots =$

$= 5^k \cdot T(\frac{n}{5^k}) + 13 \cdot 5^{k-1} \Rightarrow n \cdot 51 + 13 \cdot 5^{\log_5(n)-1} =$

$n=5^k \Leftrightarrow k=\log_5(n)$

$= n \cdot 51 + \frac{13 \cdot n}{5} = \frac{51 \cdot 5 + 13}{5} n \in \underline{\underline{O(n)}}$

b) $T(n) = \begin{cases} 3 & ; n=1 \\ 3T(\frac{n}{3}) + n & ; n=3^k \text{ for } k > 0 \end{cases}$

$T(n) = 3T(\frac{n}{3}) + n =$

$= 3(3T(\frac{n}{3^2}) + \frac{n}{3}) + n =$

$= 3^k \cdot T(\frac{n}{3^k}) + k \cdot n \Rightarrow n \cdot 3 + n \log_3(n) \in \underline{\underline{O(n \cdot \log n)}}$

$k=\log_3 n$

c) $T(n) = \begin{cases} 1 & ; n=0 \\ 2T(n-1) & ; n > 0 \end{cases}$

Let's try master method: $T(n) \leq 2 \cdot T(n-1) + 0$ $d=0$

$a=2$ $b=3$ $b \neq 1$ so we cannot use master method!

substitution method: $T(n) \leq 2 \cdot T(n-1) =$

$= 2 \cdot 2 \cdot T(n-2) = \dots$

$= 2^k \cdot T(n-k) \Rightarrow 2^n \cdot 1 \in \underline{\underline{O(2^n)}}$

$k=n$

Question 2

Note: If my writing at the bottom is too hard to read, I rewrote it below the picture.

2. Sum of array

$T(n) = \begin{cases} O(1) & ; n=1 \\ 2 \cdot T(\frac{n}{2}) + \underline{O(1)} & ; n=2^k \text{ for } k \geq 1 \end{cases}$

$\hookrightarrow$ "merging" is constant, because math operations are!

$T(n) \leq 2 \cdot T(\frac{n}{2}) + O(n^0)$

$a=2, b=2, d=0$

$\Rightarrow$ master theorem $\Rightarrow T(n) \in O(n^{\log_2 2}) = \underline{\underline{O(n)}}$

This isn't a useful application of D&C, because the complexity is $O(n)$, which is the same if we just traversed the Array, adding the numbers as we go.

The D&C is in this case worse, because it assumes that $n=2^k; k \geq 0$

EXPLANATION: This isn't a useful application of D&C, because the complexity is $O(n)$, which is the same if we just traversed the array, adding the numbers up as we go.

The D&C is worse in this case, because it assumes that $n=2^k; k \geq 0$

Question 3

```
In [6]: def slow_sort(A):
        B = [] #O(1) because size of the list is 0
        while A: #O(1) because len(list) is O(1)
            current_max = A[0] #O(1) because list slice is constant
            for element in A:
                if element > current_max:
                    current_max = element #O(n) bc this happens at most n times AKA when A is sorted increasingly
            B.append(current_max) #O(n) bc we'll need to append n elements
            A.remove(current_max) #O(n) because
        return B

example_A = [3, 5, 1, 7, 2, 4, 6, 5]
print(slow_sort(example_A))

[7, 6, 5, 5, 4, 3, 2, 1]
```

3. slow_sort $\in O(n^2)$

$T(n) = \begin{cases} 6 & ; n=1 \\ 1 \cdot T(n-1) + 2n & ; n > 1 \end{cases}$

$T(n) \leq T(n-1) + 2n =$

$= T(n-2) + 2n - 2 =$

$= T(n-k) + kn - k \Rightarrow 6 + n^2 - n \in \underline{\underline{O(n^2)}}$

$k=n$

$(n-1)$

$\underline{\underline{current_max}} \Rightarrow$ $B.append?$

Question 4

If $n = 1$; we return 0. If $n > 1$; we first divide array A of length n into 2 subarrays of length $n/2$ (assuming $n = 2^k$ for $k \geq 1$).

Each array of size $n/2$ we then further separate into $n/4$ subarrays, getting 4 arrays of size $n/4$. We keep splitting until we get to $n/2$ arrays of size 2.

At this point, we calculate the difference/distance between the two elements in each array, which means we do $n/2$ constant calculations $\rightarrow O(n)$. We also need to save the information which element is smaller and which bigger, which happens in constant time because we only need to check whether the difference is positive or negative.

We then merge back the neighbouring arrays of 2, making $n/4$ arrays of 4. At this step, we need to do $n/4 + 3$ calculations, because we only need to compare the smallest element in the 'left' array to the biggest element in the 'right' array and also the new difference to both previous differences so we can take the biggest one.

We continue in this fashion until we reach the original array, where we again compare the smallest element of the left array to the smallest element of the right array and then compare the 3 distances to each other (3 comparisons).

We can derive the following formula: $T(n) \leq 2T(n/2) + 3 \Rightarrow T(n) \in O(n)$

Question 5

(a)

We need to construct an algorithm that successfully installs a max subset of P.

For $n = 1$, we need just 1 call to the 'install' function. For $n > 1$, we first need 1 call to check whether the whole set is already a success. If it isn't, we can divide it to get two sets of length $n/2$. For each of them we again check whether they are successful or not. So we call the function twice. If one of them is successful, we can stop dividing it and only continue D&C on the other half. In case they are both unsuccessful, we keep dividing both. This process in worst case continues until we have $(d + 1)$ subsets in total or reach maximum depth $\log(n)$ (when $n = k$). Before reaching level $\log(d)$ there's no guarantee that even one subset will be successful, but after we surpass level $\log(d)$, there will be at least one subset that will yield successful. This is because in worst case scenario, there will be exactly d defected packages that are each contained in their own subset. After level $\log(d)$, there are always at most d calls to 'install' function, since if we split one subset, we again get $d + 1$ subsets in total. We continue until we reach level $\log(n)$. At that point we will call 'install' function at most d times at every level. At level $\log(n)$, each package will be their own subset, meaning they will all be defected (at most d packages). Summing to the overall complexity: $O(d * \log(n))$.

Note: whether or not d is a power of 2 does not change this reasoning.

(b)

The resulting algorithm is almost the same as the one in (a) with the main difference being that at every step before calling the 'install' function, you check whether for every $q \in Q$ there exists $p \in Q$ for subset Q . If the implication in met, you call function 'install'. If it's not, you remove all q 's from the subset and add them to a separate subset. Before you call 'install' on them, you have to do the same for all the other subsets in the level. That way you only need to call once more per level. In worst case scenario, there will be $\log(n)$ additional calls, which would mean our function would be of complexity $O(d * \log(n) + \log(n)) \in O(d * \log(n))$.